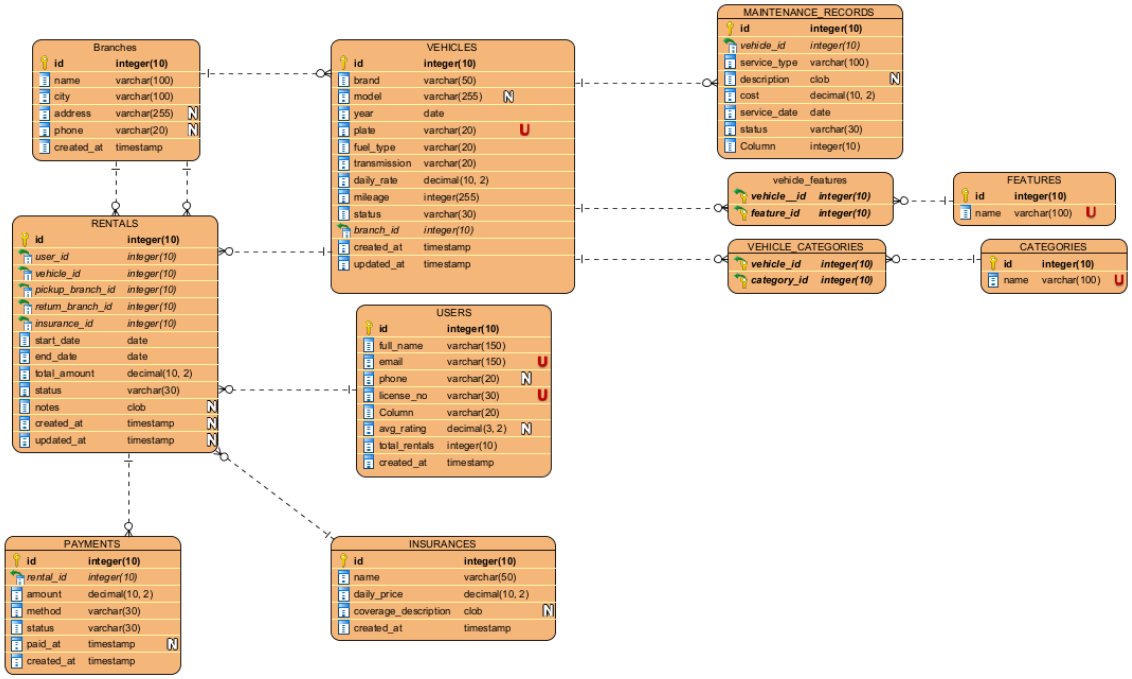
ER Diagram



Entities

• branches(id, name, city, address, phone, created_at)

• users(id, full_name, email, phone, license_no, role, avg_rating, total_rentals, created_at)

• categories(id, name)

• features(id, name)

• insurances(id, name, daily_price, coverage_description, created_at)

• vehicles(id, brand, model, year, plate, fuel_type, transmission, daily_rate, mileage, status, branch_id, created_at, updated_at)

• vehicle_categories(vehicle_id, category_id)

• vehicle_features(vehicle_id, feature_id)

• rentals(id, user_id, vehicle_id, pickup_branch_id, return_branch_id, insurance_id, start_date, end_date, total_amount, status, notes, created_at, updated_at)

• payments(id, rental_id, amount, method, status, paid_at, created_at)

• maintenance_records(id, vehicle_id, service_type, description, cost, service_date, status, created_at)

Relationships

• branch_vehicle(branch_id, vehicle_id)

• user_rental(user_id, rental_id)

• vehicle_rental(vehicle_id, rental_id)

• pickup_branch_rental(branch_id, rental_id)

• return_branch_rental(branch_id, rental_id)

• insurance_rental(insurance_id, rental_id)

• rental_payment(rental_id, payment_id)

• vehicle_maintenance(vehicle_id, maintenance_id)

• vehicle_category(vehicle_id, category_id)

• vehicle_feature(vehicle_id, feature_id)

Final Relation Schema

• branches(id, name, city, address, phone, created_at)

• users(id, full_name, email, phone, license_no, role, avg_rating, total_rentals, created_at)

• categories(id, name)

• features(id, name)

• insurances(id, name, daily_price, coverage_description, created_at)

• vehicles(id, brand, model, year, plate, fuel_type, transmission, daily_rate, mileage, status, branch_id, created_at, updated_at)

  ▪ branch_id → branches(id)

• vehicle_categories(vehicle_id, category_id)

  ▪ vehicle_id → vehicles(id)

  ▪ category_id → categories(id)

• vehicle_features(vehicle_id, feature_id)

  ▪ vehicle_id → vehicles(id)

  ▪ feature_id → features(id)

• rentals(id, user_id, vehicle_id, pickup_branch_id, return_branch_id, insurance_id, start_date, end_date, total_amount, status, notes, created_at, updated_at)

  ▪ user_id → users(id)

  ▪ vehicle_id → vehicles(id)

  ▪ pickup_branch_id → branches(id)

 ▪ return_branch_id → branches(id)

 ▪ insurance_id → insurances(id)

• payments(id, rental_id, amount, method, status, paid_at, created_at)

  ▪ rental_id → rentals(id)

• maintenance_records(id, vehicle_id, service_type, description, cost, service_date, status, created_at)

  ▪ vehicle_id → vehicles(id)

Relationship Explanation

• Each branch can have many vehicles, but each vehicle belongs to one branch.

• Each user can create many rentals, but each rental belongs to one user.

• Each vehicle can appear in many rentals over time, but each rental uses one vehicle.

• Each rental has one pickup branch and one return branch.

• Each insurance package can be used in many rentals, but each rental has one insurance package.

• Each rental can have one or more payments.

• Each vehicle can have many maintenance records.

• Vehicles and categories have a many-to-many relationship. This relationship is resolved by the vehicle_categories table.

• Vehicles and features have a many-to-many relationship. This relationship is resolved by the vehicle_features table.

Cardinality List

- branches 1:N vehicles
- users 1:N rentals
- vehicles 1:N rentals
- branches 1:N rentals as pickup_branch_id
- branches 1:N rentals as return_branch_id
- insurances 1:N rentals
- rentals 1:N payments
- vehicles 1:N maintenance_records
- vehicles M:N categories through vehicle_categories
- vehicles M:N features through vehicle_features